

Customer Churn & Revenue Intelligence Platform

A full technical walkthrough of an end-to-end analytics + ML product: synthetic Telco-style data → MySQL → SQL analytics views → scikit-learn churn/CLV models → SHAP explainability → R statistical validation → Power BI → a deployed Streamlit app — documented layer by layer, with the real numbers from the last full pipeline run.

Stack: Python · MySQL · SQL · scikit-learn · XGBoost · SHAP · R · Power BI (DAX) · Streamlit · Plotly

Dataset: 7,043 synthetic customers, structured on the IBM Telco Customer Churn dataset

Contents

1. Purpose & design philosophy	3
2. Data generation layer	3
3. Loading & validation	4
4. Database schema	4
5. SQL analytics views	5
6. ML — feature engineering	5
7. ML — churn classification & SHAP	6
8. ML — segmentation & CLV	6
9. ML — scoring & export	7
10. R statistical validation	7
11. Power BI dashboard	8
12. Streamlit application	8
13. Results recap	9
14. Design decisions & trade-offs	9

15. Known limitations

10

16. Running it end to end

10

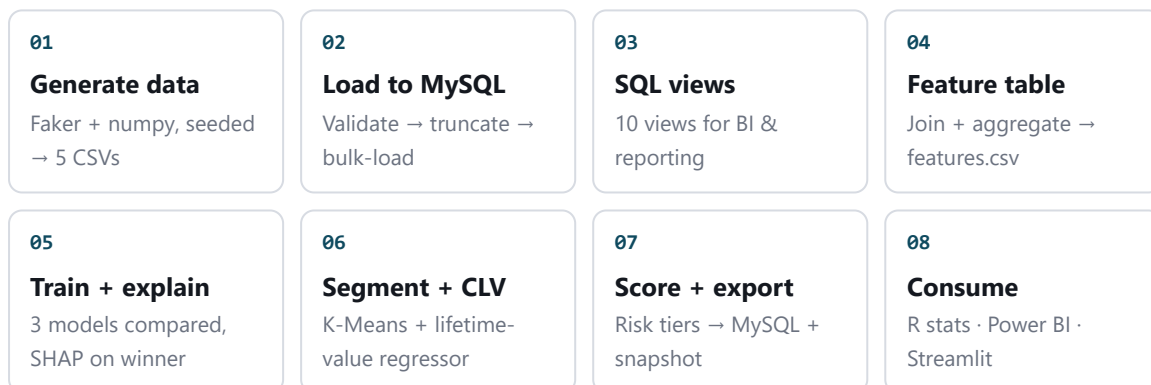
01 · PURPOSE

Purpose & design philosophy

Why this exists, and why every layer is built the way it is.

The project's job is to answer one business question end to end — *which customers are about to leave, and how much revenue is riding on them* — while touching every skill a data-analytics role typically expects: relational modeling, SQL reporting, applied ML, classical statistics, BI dashboarding, and a shippable app. The data is synthetic (nobody's real billing records are involved), but it is deliberately **not random**: it's generated with the same structure and churn drivers as the well-known IBM Telco Customer Churn dataset, so every model downstream is learning a genuine, explainable relationship rather than fitting noise. That single design choice — bake a real signal into the generator — is what lets the ML layer, the R layer, and the BI layer all independently rediscover the same three or four drivers of churn, which is the connective thread running through this whole document.

The pipeline is staged so each layer only depends on the previous layer's *output*, not its internals — CSVs feed MySQL, MySQL feeds the feature table, the feature table feeds every ML script, and the ML layer's predictions feed back into MySQL and a flat snapshot file that the deployed app can read with zero infrastructure:



Data generation layer

`etl/01_generate_data.py` · seeded with SEED = 42 for full reproducibility

Five related tables are built by five functions, each consuming the previous table's output:

build_customers()

7,043 rows. Gender is a 50/50 draw; `senior_citizen` is weighted 84/16, `partner` 52/48, `dependents` 70/30 — matching realistic telecom demographic splits rather than uniform draws. Signup dates are spread uniformly across the last 6 years (30 to 2,190 days before the reference date of 2026-07-01). City/state names come from `Faker("de_DE")`, giving the dataset a German locale consistent with the target Werkstudent market.

build_subscriptions()

The core of the dataset. Contract type is drawn 55% Month-to-month / 24% One year / 21% Two year. Tenure is then drawn from a *contract-dependent* range — month-to-month customers get 1–40 months, one-year 6–60, two-year 12–72 — so contract type and tenure are correlated the way they would be in reality (you don't see a 60-month-tenure month-to-month customer very often). Monthly charges are built additively from a base fee plus per-service add-ons (phone +€5, DSL + €24, Fiber +€42, each of 6 optional add-ons +€4–9 if present) plus Gaussian noise, clipped to €18–€130.

The churn flag itself comes from the logistic signal shown below — this is the single most important piece of the whole generator, because it is what every downstream model is trying to recover:

```
z = -0.06 * tenure_months          # longer tenure -> less likely to churn
  + 0.032 * monthly_charges        # pricier plans -> more likely to churn
  + 1.35  (if Month-to-month)      # no lock-in -> much more likely
  - 1.10  (if Two-year contract)   # locked in -> much less likely
  + 0.55  (if no tech support)
  + 0.45  (if payment = Electronic check)
  + 0.35  (if senior citizen)
  + 0.35  (if Fiber optic internet)
  + 0.20  (if no partner)
  - 3.15                                # intercept, calibrates base rate

churn_probability = 1 / (1 + e^-z)
churn_flag       = Bernoulli(churn_probability)
```

Tenure is then clipped so it never exceeds how many months could physically have passed since the customer's signup date, and churned customers get an `end_date` derived from their tenure while active customers get `end_date = NULL`.

build_transactions() / build_support_tickets() / build_churn_events()

One transaction row is generated per billing month of a subscription's tenure (146,372 rows total), with 3% of charges randomly marked failed, 2% pending, and a 1.5% chance of a partial refund row alongside any charge — this is what gives `vw_customer_clv` and the transactions table realistic billing noise rather than a perfectly clean ledger. Support tickets are drawn from a **Poisson process whose rate depends on the customer's (hidden) churn outcome** — churned customers get a base rate of 0.35 tickets/year-equivalent versus 0.15 for retained customers — which is exactly the mechanism that makes `ticket_count` such a strong predictor later (odds ratio 2.5 in the R model). Churn events are a straight 1:1 projection of churned subscriptions, with a reason sampled from Price (34%) / Service quality (24%) / Better offer (22%) / Other (10%) / Moved (10%).

TABLE	ROWS (LAST RUN)	KEY COLUMNS
customers	7,043	gender, senior_citizen, partner, dependents, signup_date, city/state/country
subscriptions	7,043	contract_type, payment_method, internet_service, 6 add-on flags, monthly_charges, tenure_months, is_active
transactions	146,372	transaction_date, amount, type (charge/refund), status (paid/failed/pending)
support_tickets	3,662	category, priority, satisfaction_score (1–5)
churn_events	2,028	churn_date, churn_reason — 28.8% overall churn rate

Loading & validation

```
etl/02_load_to_mysql.py
```

Before anything touches MySQL, every CSV is checked for a unique primary key and no nulls in its required columns (`validate()`), and referential integrity is checked in Python — every `customer_id` referenced by subscriptions, transactions, and churn_events must exist in the customers table — *before* any SQL runs, so a bad generation run fails loudly and early rather than partially loading. Loading itself truncates all six tables (with `FOREIGN_KEY_CHECKS` temporarily off, since truncation order would otherwise be blocked by the FK graph) and then bulk-inserts in FK-safe order: customers → subscriptions → transactions/support_tickets/churn_events, via SQLAlchemy's `to_sql(..., method="multi")` for efficient batched inserts.

Dependency gotcha hit during setup: MySQL 9.7 defaults new users to the `caching_sha2_password` auth plugin, which `pymysql` can only satisfy if the `cryptography` package is installed — it wasn't originally pinned in `requirements.txt`. Fixed by adding `cryptography==49.0.0` to the requirements file.

Database schema

sql/01_schema.sql — 6 normalized InnoDB tables

Five tables mirror the generated CSVs one-to-one, each foreign-keyed back to `customers.customer_id`. The sixth, `ml_predictions`, is the one table the *Python ML layer* owns and writes into — schema-defined up front so the database enforces the shape of what the model layer is allowed to write back:

```
CREATE TABLE ml_predictions (  
  customer_id      VARCHAR(20) PRIMARY KEY,  
  churn_probability DECIMAL(6,5),  
  risk_tier        VARCHAR(10),    -- Low, Medium, High  
  predicted_clv     DECIMAL(12,2),  
  segment_cluster  INT,  
  model_version     VARCHAR(20),  
  scored_at        DATETIME,  
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

Indexes are added on every foreign-key column used in joins or filters (`idx_sub_customer`, `idx_txn_customer`, `idx_txn_date`, `idx_tickets_customer`, `idx_churn_customer`) — with 146K+ transaction rows, unindexed joins against `customer_id` would otherwise force full table scans in every downstream view.

SQL analytics views

`sql/02_views.sql` — 10 views, each answering one reporting question directly in SQL rather than in application code

VIEW	ANSWERS
<code>vw_mrr_by_month</code>	MRR per calendar month, from subscriptions active during that month
<code>vw_churn_rate_by_month</code>	Churn count trend, month over month
<code>vw_churn_rate_by_contract</code>	Churn rate split by contract type
<code>vw_churn_rate_by_internet</code>	Churn rate split by internet service type
<code>vw_churn_rate_by_payment</code>	Churn rate split by payment method
<code>vw_tenure_buckets</code>	Churn rate & avg charges across 5 tenure bands (0–6, 7–12, 13–24, 25–48, 48+ mo)
<code>vw_customer_clv</code>	Realized lifetime revenue per customer, from actual paid transactions minus refunds
<code>vw_cohort_retention</code>	% of each signup-month cohort still active N months after signup
<code>vw_revenue_at_risk</code>	Monthly revenue summed by ML risk tier (joins to <code>m1_predictions</code>)
<code>vw_tickets_vs_churn</code>	Avg tickets/customer and avg satisfaction, active vs churned

Two views are worth a closer look because they demonstrate different SQL techniques:

`vw_mrr_by_month` uses a derived table of distinct calendar months (from the transaction dates) joined against subscriptions whose `[start_date, end_date]` range overlaps that month — an interval-overlap join, not a simple equi-join. `vw_revenue_at_risk` is the one view that depends on the ML layer having already run, since it joins against `m1_predictions`; if that table is empty, this view returns no rows, which is a useful sanity check that the pipeline ran in order.

Feature engineering

`ml/01_prepare_features.py`

A single SQL query joins `customers` + `subscriptions` + a per-customer aggregate subquery over `support_tickets` (count and average satisfaction, defaulting to 0 tickets / 3.0 satisfaction via `COALESCE` for customers who never filed one), pulled straight through SQLAlchemy with `pd.read_sql()`. The target column `churned` is derived as `1 - is_active`. Output: **7,043 rows × 23 columns**, saved to `data/processed/features.csv`, reused unchanged by all three remaining ML scripts.

Churn classification & SHAP

`ml/02_train_churn_models.py`

An 80/20 stratified train/test split (stratified so the ~29% churn rate is preserved in both splits) feeds a shared ColumnTransformer — OneHotEncoder on 11 categorical columns, StandardScaler on 9 numeric columns — wired into a Pipeline ahead of each of three classifiers, all trained with `class_weight="balanced"` (or XGBoost's equivalent `scale_pos_weight`) since churn is a minority class:

MODEL	ROC-AUC	PRECISION	RECALL	F1
Logistic Regression ★ winner	0.8531	0.5544	0.7906	0.6518
XGBoost	0.8467	0.5583	0.7660	0.6459
Random Forest	0.8327	0.5404	0.7906	0.6420

Logistic Regression wins on ROC-AUC despite being the simplest model — expected, since the synthetic churn signal was constructed as a literal logistic function, so a linear-in-the-logit model is close to the true data-generating process. This is also a realistic outcome in production telecom churn modeling: linear models frequently match or beat tree ensembles when the underlying drivers are additive and monotonic. The winning pipeline is serialized whole (`churn_model.joblib` — preprocessing + classifier together, so scoring new data never risks train/serve skew) alongside `model_metadata.json` recording which columns are categorical vs numeric.

Explainability: a SHAP explainer is fit on a 500-row sample of the transformed test set — `LinearExplainer` for the logistic winner, `TreeExplainer` for either tree-based alternative — producing a global summary plot (`ml/outputs/shap_summary.png`) of which features push predictions up or down across the whole population. The same explainer object is serialized and reused live inside the Streamlit app to produce a per-customer waterfall chart, so the "why" behind any single prediction is always traceable back to the same explainer that validated the model globally.

Segmentation & CLV

ml/03_segment_and_clv.py — two independent models sharing one feature table

Customer Lifetime Value

CLV here means *expected total revenue over the customer's full lifetime*, and it's modeled indirectly: a RandomForestRegressor is trained **only on already-churned customers**, learning to predict their realized tenure_months from their profile (contract, services, pricing, ticket history — tenure itself is excluded from the inputs, since that's the target). Result: **MAE $\approx$ 7.5 months, $R^2 \approx 0.14$** — reported honestly rather than inflated, because predicting exactly how long an individual customer will ultimately stay is a genuinely hard, high-variance problem; a low R^2 here is expected and typical for CLV modeling in the literature, not a bug.

That same trained regressor then predicts *expected* total tenure for still-active customers, with one constraint enforced in code — the prediction is floored at the customer's current tenure, since a customer obviously can't be expected to churn before today:

```
pred_total_tenure_active = clv_model.predict(active_customers)
pred_total_tenure_active = max(pred_total_tenure_active, active["tenure_months"])
predicted_clv = predicted_total_tenure_months * monthly_charges
```

Segmentation

K-Means with k=4 clusters on four standardized features — tenure, monthly charges, ticket count, and the predicted CLV just computed above. Raw K-Means cluster indices are arbitrary and unstable across reruns, so clusters are immediately relabeled by descending mean CLV into four fixed, meaningful names:

SEGMENT	CUSTOMERS	AVG TENURE	AVG CHARGES	AVG CLV	CHURN RATE
High-Value Loyal	1,530	36.9 mo	€70.96	€2,589	15%
Growing / Mid-Value	621	26.3 mo	€66.95	€1,766	56%
Price-Sensitive	3,240	12.0 mo	€69.87	€980	39%
Low-Value / New	1,652	19.6 mo	€24.43	€522	11%

The actionable read: **Price-Sensitive** customers pay almost as much as High-Value Loyal ones but churn at roughly 2.6× the rate on a third of the tenure — a much better retention-spend target than the cheaper Low-Value/New segment, which churns rarely simply because those customers haven't been around long enough to churn yet (survivorship, not loyalty). **Growing/Mid-Value's** 56% churn rate is the real outlier worth investigating first.

Scoring & export

`ml/04_score_and_export.py`

Every one of the 7,043 customers is scored by the saved churn pipeline and bucketed by a simple threshold function:

```
def risk_tier(p):  
    if p < 0.3: return "Low"  
    elif p < 0.6: return "Medium"  
    else:      return "High"
```

3,143

Low risk

1,489

Medium risk

2,411

High risk

€171,931

Monthly revenue at risk

Output goes to two places for two different reasons. First, `ml_predictions` in MySQL — truncated and rewritten each run — so the SQL views (`vw_revenue_at_risk`) and Power BI, which both talk to the live database, see the latest scores. Second, a flat

`data/processed/churn_predictions_snapshot.csv` — because the Streamlit app is meant to deploy to Streamlit Community Cloud, which has no access to a local/on-prem MySQL instance; the snapshot lets the deployed app work from committed files alone, with no live database dependency at all.

R statistical validation

`r_stats/churn_statistical_analysis.R` — base R only, zero CRAN dependencies

This script answers a different question than the ML layer: not "*can a model predict churn*" but "*are these relationships statistically real, individually significant, and how strong is each one on its own, independent of any ML pipeline*". It fits its own logistic regression (glm, binomial/logit link) directly on `features_scored.csv`, entirely independent of scikit-learn:

DRIVER	ODDS RATIO	95% CI	READING
Two-year contract (vs month-to-month)	0.041	0.030–0.056	~96% lower churn odds
One-year contract (vs month-to-month)	0.198	0.167–0.236	~80% lower churn odds
Support ticket count (per ticket)	2.515	2.297–2.754	each ticket more than doubles churn odds
Senior citizen	1.464	1.234–1.737	46% higher churn odds
Electronic check payment	1.527	1.279–1.824	53% higher churn odds
Fiber optic internet	1.416	1.159–1.730	42% higher churn odds
Tech support = Yes	0.595	0.506–0.701	~40% lower churn odds
Tenure (per month)	0.941	0.935–0.947	~6% lower churn odds per month stayed
Monthly charges (per €1)	1.027	1.019–1.036	modest but significant increase per euro

Every driver above is significant at $p < 0.001$. The model's overall fit — **McFadden's pseudo- $R^2 = 0.324$** — is considered a strong fit for a behavioral logit model (values above ~0.2–0.4 are generally regarded as excellent in this class of model, unlike OLS R^2 where 0.32 would look weak). Two further, simpler tests corroborate the same story from different angles: **chi-square tests of independence** confirm churn is associated with contract type ($\chi^2=1092.8$), internet service ($\chi^2=493.1$), payment method ($\chi^2=33.0$), and tech support ($\chi^2=375.5$) — all $p \ll 0.001$ — and **Welch two-sample t-tests** show churned customers pay significantly more (€68.49 vs €55.43), stay significantly less long (14.8 vs 22.8 months), and file significantly more tickets (0.77 vs 0.42) than retained customers, all at $p < 1e-49$.

Why run three separate statistical tests plus a whole ML layer, when they all point the same direction?

Because convergence across independent methods is much stronger evidence than any single method alone. If SHAP said "tenure matters" but the R odds ratios and chi-square tests disagreed, that would be a red flag for overfitting or a data leak. Here, all three tools — a different logistic regression fit in a different language, group-comparison tests with no model at all, and the ML model's own explainability output — land on the same four or five drivers. That's the actual validation.

Power BI dashboard

`powerbi/DAX_measures.txt` · `powerbi/README_dashboard_guide.md`

Power BI Desktop is GUI-only, so this layer is a guided manual build rather than a script — 20+ DAX measures (MRR, churn rate, revenue at risk, cohort retention %, average CLV by segment, etc.) written against the same SQL views described in Section 5, plus a step-by-step guide for laying out a 4-page report (Overview, Churn Drivers, Cohort/Retention, Revenue at Risk). Budgeted at roughly 2–3 hours to build by hand; screenshots are meant to be exported to `powerbi/screenshots/` once built.

Streamlit application

```
streamlit_app/app.py
```

Deliberately decoupled from MySQL — it loads only the snapshot CSV (`@st.cache_data`) and the saved model/metadata/SHAP-explainer/feature-names artifacts (`@st.cache_resource`, loaded once per server process), so the whole app runs from files committed to the repo with zero database dependency, which is what makes it deployable to Streamlit Community Cloud as-is. Four tabs:

Overview

Five headline metrics (customer count, MRR, average churn probability, revenue at risk, average CLV), a risk-tier donut chart, a churn-probability-by-contract-type bar chart, and the model comparison table plus ROC plot pulled straight from `ml/outputs/`.

High-Risk Customers

A table filterable by risk tier / contract type / minimum CLV, sorted by churn probability descending, plus a tenure-vs-churn-probability scatter sized by monthly revenue and colored by risk tier — built for a retention team to triage who to call first.

Predict Churn

A full what-if form covering every model input. On submit, it re-runs the *exact same* serialized preprocessing pipeline used in training, gets a live probability + tier, then feeds that one row through the saved SHAP explainer to render a waterfall chart — the same mechanism that produced the global summary plot in Section 7, now applied to a single hypothetical customer.

CLV & Segments

A segment summary table (customer count, avg CLV, avg tenure, avg churn probability per segment) plus a customers-per-segment bar and a CLV-vs-tenure scatter colored by segment — the visual counterpart to the segmentation table in Section 8.

Results recap

0.853

Best model ROC-AUC (Logistic Regression)

0.324

McFadden's pseudo- R^2 (R validation)

€171,931

Monthly revenue in High-risk tier

7.5 mo

CLV regressor MAE

4

Behavioral segments (K-Means)

28.8%

Overall churn rate

Design decisions & trade-offs

Why compare three models instead of just picking one?

Because "which model wins" is itself a finding worth reporting, not a foregone conclusion. Logistic Regression winning over Random Forest and XGBoost here demonstrates the underlying relationship is close to additive/linear-in-the-logit — a genuinely useful thing to know, since it means a simpler, faster, more interpretable model is also the more accurate one, which won't always be true on real (non-synthetic) churn data with genuine interaction effects.

Why decouple the app from the database entirely?

Two deployment targets have fundamentally different constraints: locally, MySQL is available and gives live, always-current predictions; on Streamlit Community Cloud, there is no persistent database at all unless you pay for one. Rather than build two separate apps, the snapshot-CSV approach lets one codebase serve both — locally you can rerun the pipeline and refresh the snapshot; deployed, the snapshot simply becomes the dataset the whole internet sees.

Why base R with no CRAN packages?

Removes an entire class of setup friction (no `install.packages()` step, nothing that can fail due to CRAN mirror issues or missing system libraries like `libcurl`) while still demonstrating real inferential statistics — `glm()`, `chisq.test()`, and `t.test()` are all in R's default distribution.

Why report a weak CLV R^2 instead of engineering it away?

Because it's true, and reporting it honestly is itself a signal of statistical maturity that the README explicitly calls out. CLV prediction is a hard problem in the literature generally — individual customer lifetimes have high inherent variance that no amount of feature engineering fully removes — and claiming a suspiciously high R^2 here would be a bigger red flag to a technical reviewer than an honest, modest one.

Known limitations

- **Synthetic data ceiling:** because churn is generated from a known logistic formula, ROC-AUC and McFadden's R^2 are both structurally higher than they would be on messier real-world data with unmeasured confounders — these numbers demonstrate methodology, not a claim about real-world telecom churn rates.
- **No hyperparameter tuning:** all three classifiers use fixed, reasonable defaults (e.g. `RandomForestClassifier(n_estimators=300, max_depth=8)`) rather than grid/random search — a deliberate scope cut to keep the pipeline fast and reproducible, not an oversight, but a next step for squeezing further performance.
- **Single train/test split, no cross-validation:** the reported metrics come from one 80/20 split; a k-fold CV would give a confidence interval around ROC-AUC rather than a point estimate.
- **Monthly-resolution time granularity:** transactions and tenure are modeled in whole months, so the platform can't answer sub-monthly questions (e.g. time-of-month billing-failure patterns).
- **CLV model has no confidence intervals:** `predicted_clv` is a point estimate from a Random Forest regressor with $R^2 \approx 0.14$ — it should be read directionally (segment comparisons) rather than trusted as an exact euro figure per customer.

Running it end to end

```
# 1. Environment
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt

# 2. MySQL: create DB + user, then schema
mysql -u root -p -e "CREATE DATABASE churn_platform; CREATE USER 'churn_app'@'localhost'
IDENTIFIED BY 'yourpassword'; GRANT ALL ON churn_platform.* TO 'churn_app'@'localhost';"
mysql -u churn_app -p churn_platform < sql\01_schema.sql
# fill in .env with DB credentials

# 3. ETL
.venv\Scripts\python etl\01_generate_data.py
.venv\Scripts\python etl\02_load_to_mysql.py
mysql -u churn_app -p churn_platform < sql\02_views.sql

# 4. ML (run in order – each script depends on the previous one's output)
cd ml
..\venv\Scripts\python 01_prepare_features.py
..\venv\Scripts\python 02_train_churn_models.py
..\venv\Scripts\python 03_segment_and_clv.py
..\venv\Scripts\python 04_score_and_export.py
cd ..

# 5. R statistical validation (run from project root)
Rscript r_stats\churn_statistical_analysis.R

# 6. Run the app locally
.venv\Scripts\streamlit run streamlit_app\app.py
```

Two gotchas hit during a real setup on Windows: (1) MySQL 9.7's default `caching_sha2_password` auth needs the cryptography package for `pymysql` — now pinned in `requirements.txt`. (2) Streamlit's first-run "onboarding email" prompt blocks a non-interactive/background launch entirely; either answer it once interactively, pre-create `~/.streamlit/credentials.toml` with an empty email, or pass `--server.headless true`.